

♥ Like

📌 Save

🔗 Share



@thedevspaceio

JavaScript Arrays Cheatsheet



Array properties and methods

Method	Mutating	Description
<code>Array.length</code>		Returns the number of elements in the array.
<code>Array.isArray()</code>		Checks if a value is an array.
<code>Array.from()</code>		Creates an array from an iterable or array-like object.
<code>Array.of()</code>		Creates an array from the given arguments.
<code>.push()</code>	Yes	Adds elements to the end.
<code>.pop()</code>	Yes	Removes the last element.
<code>.unshift()</code>	Yes	Adds elements to the start.
<code>.shift()</code>	Yes	Removes the first element.
<code>.concat()</code>		Merges arrays into a new array.
<code>.slice()</code>		Returns a shallow copy of a portion.
<code>.splice()</code>	Yes	Adds/removes elements.
<code>.indexOf()</code>		Returns the first index of a value.
<code>.lastIndexOf()</code>		Returns the last index of a value.
<code>.includes()</code>		Checks if a value exists.
<code>.find()</code>		Returns the first matching element.

Method	Mutating	Description
<code>.findIndex()</code>		Returns the index of the first matching element.
<code>.findLast()</code>		Returns the last matching element.
<code>.findLastIndex()</code>		Returns the index of the last matching element.
<code>.filter()</code>		Returns a new array of matching elements.
<code>.map()</code>		Returns a new array of transformed elements.
<code>.reduce()</code>		Reduces the array to a single value.
<code>.reduceRight()</code>		Reduces from right to left.
<code>.every()</code>		Returns <code>true</code> if all elements match.
<code>.some()</code>		Returns <code>true</code> if any element matches.
<code>.forEach()</code>		Runs a function for each element.
<code>.sort()</code>	Yes	Sorts the array.
<code>.reverse()</code>	Yes	Reverses the array.
<code>.join()</code>		Joins elements into a string.
<code>.flat()</code>		Flattens nested arrays.
<code>.flatMap()</code>		Maps then flattens one level.
<code>.fill()</code>	Yes	Fills elements with a value.
<code>.at()</code>		Returns element at index, supports negatives.

Creating arrays

Array.isArray()

Checks if a value is an array.

js



```
1 Array.isArray([1, 2, 3]); // true
2 Array.isArray("hello"); // false
```

Array.from()

Creates an array from an iterable or array-like object.

js



```
1 Array.from("hello"); // ["h", "e", "l", "l", "o"]
2 Array.from({ length: 3 }, (_, i) => i); // [0, 1, 2]
```

Array.of()

Creates an array from the given arguments.

js



```
1 Array.of(1, 2, 3); // [1, 2, 3]
2 Array.of(7); // [7]
```

Adding and removing elements

`.push()` / `.pop()`

Add or remove elements at the end. Both mutate the array.

js



```
1 const arr = [1, 2, 3];
2 arr.push(4); // [1, 2, 3, 4]
3 arr.pop(); // 4
```

`.unshift()` / `.shift()`

Add or remove elements at the start. Both mutate the array.

js



```
1 const arr = [2, 3];
2 arr.unshift(1); // [1, 2, 3]
3 arr.shift(); // 1
```

`.splice()`

Adds or removes elements at a specific index. Mutates the array.

js



```
1 const arr = [1, 2, 3, 4];
2 arr.splice(1, 2); // removes 2 elements starting at index 1
3 // arr is now [1, 4]
4
5 arr.splice(1, 0, 2, 3); // inserts 2 and 3 at index 1
6 // arr is now [1, 2, 3, 4]
```

Transforming arrays

`.map()`

Returns a new array with each element transformed.

js



```
1 [1, 2, 3].map((x) => x * 2); // [2, 4, 6]
```

`.filter()`

Returns a new array of elements that pass the test.

js



```
1 [1, 2, 3, 4].filter((x) => x % 2 === 0); // [2, 4]
```

`.reduce()`

Reduces the array to a single value.

js



```
1 [1, 2, 3, 4].reduce((sum, value) => sum + value, 0); // 10
```

Searching arrays

`.includes()`

Checks if a value exists.

js



```
1 [1, 2, 3].includes(2); // true
2 [1, 2, 3].includes(5); // false
```

`.find()` / `.findIndex()`

Find the first matching element (`.find()`) or its index (`.findIndex()`).

js



```
1 [1, 2, 3].find((x) => x > 1); // 2
2 [1, 2, 3].findIndex((x) => x > 1); // 1
```


Iterating arrays

`.forEach()`

Runs a function for each element.

js



```
1 [1, 2, 3].forEach((x) => console.log(x));
```

`.every()`

Returns `true` if all elements pass the test.

js



```
1 [2, 4, 6].every((x) => x % 2 === 0); // true
2 [1, 2, 3].every((x) => x > 0); // true
```

`.some()`

Returns `true` if at least one element passes the test.

js



```
1 [1, 2, 3].some((x) => x > 2); // true
2 [1, 2, 3].some((x) => x > 5); // false
```

Ordering and joining

`.sort()`

Sorts the array in place. Mutates the array.

js



```
1 [3, 1, 2].sort((a, b) => a - b); // [1, 2, 3]
```

`.reverse()`

Reverses the array in place. Mutates the array.

js



```
1 [1, 2, 3].reverse(); // [3, 2, 1]
```

`.join()`

Joins elements into a string.

js



```
1 ["a", "b", "c"].join("-"); // "a-b-c"
```

Flattening arrays

`.flat()`

Flattens nested arrays.

js



```
1 [1, [2, 3], [4, [5]]].flat(); // [1, 2, 3, 4, [5]]
2 [1, [2, [3]]].flat(2); // [1, 2, 3]
```

`.flatMap()`

Maps each element then flattens the result one level.

js



```
1 [1, 2, 3].flatMap((x) => [x, x * 2]); // [1, 2, 2, 4, 3, 6]
```